

GRADO EN INGENIERÍA INFORMÁTICA DE GESTIÓN Y SISTEMAS DE INFORMACIÓN

TRABAJO DE FIN DE GRADO

**CONVERSIÓN DE ACTAS DEL AYUNTAMIENTO DE
BILBAO A KNOWLEDGE GRAPHS (RDF, OWL,
SPARQL, SHACL) Y VECTORIAL MEDIANTE LLMS,
ENLAZADO CON WIKIDATA Y OFRECIENDO
FUNCIONALIDAD RAG. INTERFAZ WEB: VISUAL,
CONSULTAS, CONVERSACIONAL**



Alumno: Lopez, Sanchez, Peio

Director: Egaña Aranguren, Mikel

Curso: 2026-2027

Fecha: <XXXX, día, mes, año>

Resumen Laburpena Abstract

Resumen:

Este trabajo aborda la transformación de las actas del Pleno del Ayuntamiento de Bilbao (2007–2026), publicadas en formato PDF, en dos sistemas complementarios de recuperación de información consultables en lenguaje natural: un RAG vectorial y un sistema GraphRAG. A partir de la descarga automatizada del portal municipal, se extraen de forma estructurada las proposiciones individuales junto con sus metadatos esenciales: fecha de la sesión, proponente, tema, entidades, página exacta y el resultado de la votación –tanto agregado por grupo como nominal por concejal cuando consta en el acta–. Esta información nutre un grafo de conocimiento RDF/OWL que modela la actividad plenaria, combinando extracción determinista para garantizar la fidelidad de los votos con modelos de lenguaje para la clasificación temática y el reconocimiento de entidades.

Sobre este corpus de 3.422 proposiciones se implementan dos arquitecturas de consulta: un RAG vectorial con búsqueda híbrida multicanal y reordenación semántica (reranking), y un sistema GraphRAG que traduce el lenguaje natural a consultas SPARQL para resolver preguntas de agregación y conteo mediante razonamiento automático. Ambos sistemas garantizan la trazabilidad citando la página exacta del documento original y se integran en una interfaz conversacional construida con Chainlit. El trabajo concluye con una evaluación comparativa sistemática sobre un conjunto de preguntas de referencia contrastadas exhaustivamente contra los documentos oficiales.

Palabras Clave: GraphRAG, RAG vectorial, grafo de conocimiento, SPARQL, LLM, datos abiertos municipales.

Laburpena:

Lan honek Bilboko Udalaren Osoko Bilkuraren aktak (2007–2026), PDF formatuan argitaratuak, hizkuntza naturalean kontsultatu daitezkeen informazio-berreskurapeneko bi sistema osagarritan –RAG bektorial bat eta GraphRAG sistema bat– eraldatzeari ekiten dio. Udalaren atarian modu automatizatuan deskargatutako aktetatik, egiturazko moduan ateratzen dira proposamen individualak, beren funtsezko metadatuekin batera: bilkuraren data, proposatzailea, gaia, entitateak, orrialde zehatza eta bozketaren emaitza –taldeka metatuta zein, aktan hala jasotzen denean, zinegotziz zinegotzi–. Informazio horrek RDF/OWL ezagutza-grafo bat elikatzen du, osoko bilkuraren jardura modelatzen duena, bozen zehaztasuna bermatzeko atzemate determinista eta gai-sailkapenerako eta entitateen ezagutzarako hizkuntza-ereduak konbinatuz.

3.422 proposamenez osatutako corpus horren gainean, bi kontsulta-arkitektura ezartzen dira: bilaketa hibrido anizkanaleko eta birsailkapen semantikoko (reranking) RAG bektorial bat, eta hizkuntza naturala SPARQL kontsultetara itzultzen duen GraphRAG sistema bat, arrazoiketa automatikoaren bidez metaketa- eta zenbaketa-galderak ebazteko. Bi sistemek

jatorrizko dokumentuaren orrialde zehatza aipatuz bermatzen dute trazabilitatea, eta Chainlit-ekin eraikitako elkarriketa-interfaze batean integratzen dira. Lana jatorrizko dokumentuen aurka zorrotz egiaztatutako erreferentzia-galdera multzo baten gaineko ebaluazio konparatibo sistematiko batekin amaitzen da.

Gako-hitzak: GraphRAG, RAG bektoriala, ezagutza-grafoa, SPARQL, LLM, udal datu irekiak.

Abstract:

This work addresses the transformation of the minutes of the Plenary Session of the Bilbao City Council (2007–2026), published in PDF format, into two complementary natural-language information retrieval systems: a vector RAG and a GraphRAG system. From documents downloaded automatically from the municipal portal, individual proposals are extracted in structured form together with their essential metadata: session date, proposer, topic, entities, exact page, and the outcome of the vote –both aggregated by political group and, when explicitly recorded in the minutes, nominal by councillor–. This information feeds an RDF/OWL knowledge graph that models plenary activity, combining deterministic extraction to guarantee the fidelity of the votes with language models for topic classification and entity recognition.

Two query architectures are implemented over this corpus of 3,422 proposals: a vector RAG with multi-channel hybrid search and semantic reranking, and a GraphRAG system that translates natural language into SPARQL queries to resolve aggregation and counting questions through automated reasoning. Both systems guarantee traceability by citing the exact page of the original document and are exposed through a conversational web interface built with Chainlit. The work concludes with a systematic comparative evaluation over a set of reference questions verified exhaustively against the original documents.

Keywords: GraphRAG, vector RAG, knowledge graph, SPARQL, LLM, open municipal data.

Índice

Resumen Laburpena Abstract	1
Lista de figuras	6
Lista de tablas	7
Lista de acrónimos	9
Empleo de Inteligencia Artificial (IA)	10
1. Introducción	11
1.1. Motivación	11
1.2. Objetivos y alcance	12
1.3. Conexión con los Objetivos de Desarrollo Sostenible	13
1.4. Estructura de la memoria	14
2. Contexto	15
2.1. Transparencia y digitalización de la administración pública	15
2.2. El auge de los LLMs	15
2.3. RAG	16
2.4. Grafos de Conocimiento	17
2.5. GraphRAG	17
2.6. El Pleno del Ayuntamiento de Bilbao	17
3. Descripción de requerimientos y análisis del estado del arte	19
3.1. Requisitos del sistema	19
3.2. Estado del arte de las tecnologías utilizadas	20
4. Planificación del proyecto	22

4.1. Diagrama EDT	22
4.2. Descripción de tareas	22
4.3. Gestión del tiempo	27
4.4. Metodología de desarrollo	28
4.5. Evaluación económica	28
4.6. Análisis de riesgos	29
5. Análisis de alternativas	33
5.1. Modelos de lenguaje: local (Ollama) frente a API en la nube (Groq / Gemini)	33
5.2. Almacenamiento del grafo: rdflib/OWL-RL frente a una base de grafos dedicada (p.ej. GraphDB, Neo4j)	34
5.3. Almacén vectorial y recuperación: ChromaDB e híbrida (semántica + literal + temática) frente a solo semántica	35
6. Descripción de la solución propuesta	36
6.1. Arquitectura general del sistema	36
6.2. Construcción del grafo de conocimiento	36
6.3. Sistema de recuperación de información	36
6.4. Generación de respuestas	36
6.5. Interfaz web del sistema	36
6.6. Flujo completo del sistema	36
6.7. Configuración y decisiones de implementación	36
7. Corpus y preprocesado de datos	37
7.1. El corpus de actas del Pleno	37
7.2. Pipeline de preprocesado	37
8. Análisis de resultados	38
8.1. Objetivo de la evaluación	38
8.2. Metodología	38
8.3. Resultados globales	40
8.4. Análisis sobre la recuperación	41
8.5. Análisis sobre la generación	42
8.6. Comparativa GraphRAG frente a RAG vectorial	43
8.7. Configuración final y valoración global	44

8.8. Limitaciones	44
9. Conclusiones y trabajo futuro	46
9.1. Conclusiones del trabajo	46
9.2. Cumplimiento de objetivos	46
9.3. Conclusiones sobre las pruebas realizadas	46
9.4. Contribuciones científico-técnicas	46
9.5. Trabajo futuro	46
Bibliografía	47

Lista de figuras

1.	Icono del Objetivo 16: Paz, justicia e instituciones sólidas	14
2.	Icono del Objetivo 11: Ciudades y comunidades sostenibles	14
3.	Fachada del Ayuntamiento de Bilbao	18
4.	Diagrama EDT del proyecto	22
5.	Diagrama de Gantt del proyecto (semanas desde el 22 de enero de 2026) . .	27

Lista de tablas

1.	Requisitos funcionales del sistema	19
2.	Requisitos no funcionales del sistema	20
3.	Paquete de trabajo: 1. Gestión y planificación	23
4.	Paquete de trabajo: 1.1. Reuniones con el director	23
5.	Paquete de trabajo: 1.2. Gestión del tiempo	23
6.	Paquete de trabajo: 2. Investigación y estudio previo	23
7.	Paquete de trabajo: 2.1. Estudio del estado del arte	23
8.	Paquete de trabajo: 2.2. Análisis de alternativas tecnológicas	24
9.	Paquete de trabajo: 3. Desarrollo del sistema	24
10.	Paquete de trabajo: 3.1. Extracción y preprocesado del corpus	24
11.	Paquete de trabajo: 3.2. RAG vectorial	24
12.	Paquete de trabajo: 3.3. GraphRAG	25
13.	Paquete de trabajo: 3.4. Interfaz web	25
14.	Paquete de trabajo: 4. Evaluación del sistema	25
15.	Paquete de trabajo: 4.1. Diseño de la batería de preguntas de referencia	25
16.	Paquete de trabajo: 4.2. Evaluaciones y comparativa	26
17.	Paquete de trabajo: 4.3. Validación contra PDF real y regresión	26
18.	Paquete de trabajo: 5. Documentación	26
19.	Paquete de trabajo: 5.1. Redacción de la memoria	27
20.	Paquete de trabajo: 5.2. Documentación técnica	27
21.	Coste de mano de obra	28
22.	Coste de hardware (amortizado)	29
23.	Coste de software	29
24.	Costes indirectos (electricidad)	29
25.	Coste total del proyecto	30

26.	Riesgo: 1. Dependencia de APIs externas	30
27.	Riesgo: 2. Errores silenciosos en la generación de SPARQL	30
28.	Riesgo: 3. Calidad variable del OCR en actas antiguas	31
29.	Riesgo: 4. Plazo de entrega ajustado con un único desarrollador	31
30.	Riesgo: 5. Nombres y grupos ambiguos o mal atribuidos en las actas	32
31.	Métricas RAGAS del RAG vectorial en su configuración final (generador local Qwen2.5-7B)	40
32.	Métricas RAGAS antes y después de corregir la selección de palabras clave del canal literal	42
33.	Experimento controlado: mismo contexto recuperado, distinto generador	43

Lista de acrónimos

LLM Large Language Model

RAG Retrieval Augmented Generation

GraphRAG Graph Retrieval Augmented Generation

RDF Resource Description Framework

OWL Web Ontology Language

SPARQL SPARQL Protocol and RDF Query Language

SKOS Simple Knowledge Organization System

SHACL Shapes Constraint Language

LOD Linked Open Data

API Application Programming Interface

SQL Structured Query Language

GPU Graphics Processing Unit

VRAM Video Random Access Memory

ODS Objetivos de Desarrollo Sostenible

EDT Estructura de Desglose del Trabajo

PDF Portable Document Format

OCR Optical Character Recognition

TFG Trabajo Fin de Grado

Empleo de Inteligencia Artificial (IA)

1. Introducción

El portal de actas del Ayuntamiento de Bilbao publica, para cada sesión del Pleno desde 2007 hasta la actualidad, varios documentos PDF distintos: el orden del día, extractos, el resumen de la sesión y el acta completa. De estos, el acta es el único que recoge de forma íntegra las proposiciones, el debate y el resultado de cada votación, por lo que es el documento que utiliza este trabajo; cada acta puede superar el centenar de páginas, y el corpus completo reúne cerca de veinte años de actas y 3.422 proposiciones distintas.

Este capítulo presenta la motivación que ha llevado al desarrollo del proyecto, los objetivos planteados, su relación con los Objetivos de Desarrollo Sostenible y la estructura del resto de la memoria.

1.1. Motivación

Dentro de cada acta, el contenido relevante –las proposiciones y sus votaciones– se mezcla con una cantidad considerable de contenido puramente procedimental: listas de asistentes, fórmulas de quórum, aprobación del acta anterior, cuestiones de orden o informes que solo se dan por conocidos sin votación. Identificar qué parte de cada acta es efectivamente una proposición –y separarla del resto– es en sí mismo parte del problema a resolver.

A esto se suma un problema práctico adicional: encontrar información concreta –qué votó un grupo político sobre un tema, cuántas proposiciones sobre vivienda se presentaron en un año determinado, cómo ha evolucionado un asunto a lo largo de los mandatos– exige leer manualmente decenas de documentos extensos, o como mínimo saber de antemano en qué fecha y página buscar. No existe ninguna vía para hacer preguntas en lenguaje natural sobre este archivo y obtener una respuesta verificable, con cifras y citas a la fuente original.

1.1.1. A quién beneficia el sistema

El sistema desarrollado tiene utilidad directa para varios perfiles distintos de usuario, cada uno con una forma de preguntar y un tipo de respuesta que le interesa más:

Ciudadanía Puede preguntar en lenguaje natural por el estado o el resultado de una propuesta concreta –p. ej. “¿qué se votó sobre el polideportivo de mi barrio?”– sin necesidad de conocer de antemano en qué acta ni en qué página buscar, y obtener la cita exacta al documento original como respaldo.

Periodistas Se benefician sobre todo de las preguntas de agregación y comparación entre

grupos políticos –p. ej. cuántas propuestas sobre vivienda ha presentado cada partido en un mandato, o cómo ha votado cada grupo un asunto concreto–, el tipo de consulta en el que el sistema GraphRAG aporta más valor frente a una búsqueda de texto convencional (Capítulo 2).

Personal técnico municipal Puede recuperar rápidamente el precedente de un asunto ya tratado en plenos anteriores –qué se acordó, qué grupo lo propuso, si se aprobó con enmienda– sin depender de la memoria de quien estuvo presente en aquella sesión ni de una búsqueda manual en el archivo.

Investigadores Disponen de un corpus estructurado y consultable de forma sistemática –por tema, por grupo, por periodo temporal– que permite estudiar la actividad plenaria a lo largo de casi veinte años sin procesar manualmente cada acta en PDF.

1.2. Objetivos y alcance

El objetivo general de este TFG es cerrar la brecha descrita en la motivación: construir un sistema que permita consultar en castellano, mediante preguntas formuladas de forma natural, el contenido íntegro de las actas del Pleno, y que responda con datos verificables –enlazados a la página exacta del PDF de origen– en lugar de con texto generado sin respaldo documental.

1.2.1. Objetivos específicos

Del objetivo general se derivan los siguientes objetivos específicos:

1. Extraer de forma automatizada las proposiciones individuales de cada acta – junto con su fecha, grupo o particular proponente, tema, entidades mencionadas y resultado de la votación– a partir del corpus de PDF publicado por el Ayuntamiento (Capítulo 7).
2. Construir un grafo de conocimiento RDF/OWL que modele las proposiciones, los grupos políticos, los concejales y los temas, y que permita razonamiento automático sobre relaciones temáticas jerárquicas (Capítulo 6).
3. Implementar un sistema de recuperación de información basado en búsqueda vectorial híbrida (semántica, literal y temática) sobre el mismo corpus.
4. Implementar un segundo sistema de recuperación, GraphRAG, que traduzca la pregunta del usuario a una consulta SPARQL y la ejecute sobre el grafo de conocimiento, para responder preguntas de agregación y razonamiento multi-salto que el sistema vectorial no puede resolver con precisión.
5. Exponer ambos sistemas mediante una interfaz web conversacional que cite, para cada dato mostrado, el documento y la página exacta del PDF de origen.
6. Evaluar de forma comparativa ambos sistemas sobre un conjunto de preguntas de referencia con respuesta verificada a mano contra las actas originales, para caracterizar en qué tipo de preguntas destaca cada arquitectura (Capítulo 8).

1.2.2. Alcance del sistema desarrollado

El sistema cubre el corpus completo de actas del Pleno publicadas por el Ayuntamiento de Bilbao entre 2007 y 2026 (3.422 proposiciones), con extracción de metadatos, clasificación temática, dos sistemas de recuperación de información independientes y una interfaz web conversacional común a ambos.

Quedan fuera del alcance de esta entrega:

- El enlazado del grafo de conocimiento con Wikidata para desambiguar y enriquecer entidades no personales (empresas, lugares, organismos). Se valoró como extensión natural del grafo RDF/OWL, pero se descartó para esta entrega por el coste de desambiguación fiable que exige –en particular, distinguir entidades homónimas sin introducir enlaces incorrectos– y queda recogido como trabajo futuro (Capítulo 9).
- La desambiguación por género de concejales homónimos en la capa de voto nominal, más allá de la resolución por apellido ya implementada (Capítulo 7).
- Una evaluación con usuarios reales del sistema: la evaluación de este trabajo es exclusivamente técnica, sobre una batería de preguntas de referencia verificada contra las actas originales (Capítulo 8), sin sesiones de uso con los perfiles reales identificados en la Sección 1.1.1.

1.3. Conexión con los Objetivos de Desarrollo Sostenible

Con el objetivo de conseguir un futuro sostenible en el que se haya erradicado la desigualdad, se proteja el medioambiente y se garantice el bienestar de todos, los estados miembros de las Naciones Unidas aprobaron 17 objetivos el 25 de septiembre de 2015, con el propósito de cumplirlos para el año 2030.

El Objetivo de Desarrollo Sostenible (ODS) 16, *Paz, justicia e instituciones sólidas*, busca promover sociedades pacíficas e inclusivas, facilitar el acceso a la justicia para todos y construir instituciones eficaces, responsables e inclusivas a todos los niveles. En particular, su meta 16.10 persigue garantizar el acceso público a la información y proteger las libertades fundamentales, de acuerdo con la legislación nacional y los acuerdos internacionales.

El presente TFG se alinea con este objetivo al desarrollar una herramienta que convierte un archivo público municipal –las actas del Pleno del Ayuntamiento de Bilbao– en una fuente consultable en lenguaje natural, con cifras y citas verificables a la fuente original. Aunque las actas ya son públicas, su volumen (cerca de veinte años, más de un centenar de páginas por sesión) y su formato PDF no estructurado las hacen, en la práctica, de difícil acceso para quien no dispone del tiempo o la pericia necesarios para revisarlas manualmente. El sistema desarrollado reduce esa asimetría de información y facilita el escrutinio de la actividad municipal por parte de la ciudadanía, la prensa y la investigación, tal como se detalla en la Sección 1.1.1.

En segundo término, el trabajo conecta también con el ODS 11, *Ciudades y comunidades sostenibles*, cuya meta 11.3 promueve una urbanización inclusiva y participativa. Un acceso más directo al historial de decisiones del Pleno –qué se propuso, quién lo propuso y qué se decidió– es una condición previa para que la ciudadanía pueda participar de forma informada en los asuntos municipales, más allá del mero acceso a la información que cubre el ODS 16.



Figura 1: Icono del Objetivo 16: Paz, justicia e instituciones sólidas



Figura 2: Icono del Objetivo 11: Ciudades y comunidades sostenibles

1.4. Estructura de la memoria

Esta memoria se organiza como sigue. El Capítulo 2 introduce los conceptos técnicos en los que se apoya el sistema (modelos de lenguaje, RAG, grafos de conocimiento y GraphRAG) y describe el entorno institucional del Pleno del Ayuntamiento de Bilbao. El Capítulo 3 recoge los requisitos del sistema y el estado del arte de las tecnologías empleadas. El Capítulo 4 recoge la planificación temporal seguida, la evaluación económica y el análisis de riesgos del proyecto. El Capítulo 5 analiza las principales alternativas tecnológicas evaluadas antes de fijar la arquitectura final. El Capítulo 6 describe en detalle la solución desarrollada: los dos sistemas de recuperación de información contruidos –uno basado en búsqueda vectorial y otro en un grafo de conocimiento RDF/OWL consultado por SPARQL– y la interfaz web conversacional que los expone al usuario. El Capítulo 7 describe el corpus de actas de partida y el pipeline que lo transforma en proposiciones estructuradas. El Capítulo 8 presenta la evaluación comparativa de ambos sistemas. La memoria cierra con las conclusiones y los anexos.

2. Contexto

En este capítulo se presenta el contexto en el que se enmarca el trabajo, se introducen los conceptos de LLM, RAG, grafos de conocimiento y GraphRAG, que constituyen la base técnica del sistema desarrollado, y se describe el entorno institucional –el Pleno del Ayuntamiento de Bilbao– sobre el que se aplica.

2.1. Transparencia y digitalización de la administración pública

La transparencia y el acceso a la información pública son, desde hace más de una década, un objetivo explícito de la administración española. La Ley 19/2013, de 9 de diciembre, de transparencia, acceso a la información pública y buen gobierno [1], obliga a las administraciones públicas a publicar de forma activa la información relativa a su actividad institucional. En paralelo, el movimiento de *open government* promueve que esa información no solo se publique, sino que se haga accesible y reutilizable, en vez de limitarse a cumplir el mínimo legal de publicación.

En la práctica, sin embargo, publicar un documento no equivale a hacerlo accesible. Los portales de transparencia municipales publican de forma sistemática actas, órdenes del día y resúmenes de sesión en formato PDF, un formato pensado para la lectura secuencial y no para la consulta puntual ni el análisis agregado. Cuando el volumen de documentos publicados crece –como ocurre con casi veinte años de actas del Pleno de una ciudad como Bilbao– esa distancia entre “publicado” y “accesible” se hace más evidente: la información está disponible en sentido legal, pero encontrar un dato concreto dentro de ella exige un esfuerzo de búsqueda manual que en la práctica limita quién puede efectivamente usarla.

En este contexto, las técnicas de inteligencia artificial que permiten consultar documentación extensa en lenguaje natural –y en particular las técnicas RAG y GraphRAG que se presentan en las siguientes secciones– ofrecen una vía para cerrar esa distancia entre publicación y acceso real, aplicada aquí a un caso concreto de documentación administrativa pública.

2.2. El auge de los LLMs

Los modelos de lenguaje de gran tamaño, conocidos como LLM por sus siglas en inglés *Large Language Models*, son modelos de inteligencia artificial entrenados sobre corpus masivos de texto que, dada una secuencia de entrada, predicen la continuación más probable. Esta capacidad, combinada con un ajuste posterior para seguir instrucciones, les permite realizar tareas tan diversas como responder preguntas, resumir documentos o

mantener una conversación con el usuario a partir de una instrucción (*prompt*).

El interés generalizado por los LLM se intensificó a partir de noviembre de 2022, cuando OpenAI presentó ChatGPT [4], una interfaz conversacional que popularizó el uso de estos modelos entre el público general. Desde entonces, la oferta de LLM disponibles –tanto propietarios como de código abierto– se ha ampliado considerablemente, lo que permite hoy elegir entre modelos servidos como API en la nube o modelos más pequeños ejecutables en local.

Un LLM, sin embargo, no tiene por sí mismo acceso a información posterior a su entrenamiento ni a documentos privados o muy específicos –como las actas de un ayuntamiento concreto–, y tampoco garantiza que lo que genera sea correcto: puede producir afirmaciones plausibles pero falsas, fenómeno conocido como *alucinación* [5], especialmente relevante en un contexto donde se maneja información institucional que debe ser verificable. A diferencia de un escenario con documentación empresarial confidencial, en este trabajo las actas del Pleno son de por sí públicas, por lo que la motivación para complementar el LLM con una fuente documental externa no es la confidencialidad, sino la precisión y la trazabilidad de cada dato: la respuesta debe poder verificarse contra el documento original, algo que el conocimiento interno del modelo no puede ofrecer por sí solo. Ambas limitaciones –la falta de acceso a documentación específica y la falta de verificabilidad– son las que motivan la técnica que se presenta a continuación.

2.3. RAG

La generación aumentada por recuperación, conocida como RAG por sus siglas en inglés *Retrieval-Augmented Generation* [2], es una técnica que combina un LLM con un mecanismo de recuperación de información, con el objetivo de que el modelo genere respuestas apoyándose en información obtenida de una base de conocimiento externa en lugar de únicamente en lo aprendido durante su entrenamiento.

El funcionamiento de un sistema RAG puede dividirse en dos fases. En la fase de preparación se construye la base de conocimiento: los documentos se procesan y se dividen en fragmentos más pequeños (*chunks*), que se transforman en representaciones vectoriales (*embeddings*) –números que capturan el significado del texto, de forma que fragmentos semánticamente similares quedan cerca en el espacio vectorial– y se almacenan en una base de datos vectorial. En la fase de consulta, la pregunta del usuario se transforma también en un *embedding*, se compara con los almacenados para recuperar los fragmentos más relevantes, y estos se incorporan al *prompt* que recibe el LLM para generar la respuesta.

Esta arquitectura tiene dos ventajas directas para este proyecto: permite responder sobre un corpus privado y actualizable (las actas del Pleno) sin reentrenar ningún modelo, y permite citar la fuente exacta de cada afirmación, ya que se conoce qué fragmentos –y por tanto qué documento y página– sustentan la respuesta generada.

La variante más directa de RAG se conoce como *Naive RAG*, que sigue exactamente el esquema de dos fases descrito arriba sin ningún refinamiento adicional. Sobre esa base existen variantes más avanzadas [10]: *Advanced RAG* introduce mejoras en las distintas fases del proceso –como la reformulación de la consulta o la reordenación de los fragmentos recuperados– sin cambiar la arquitectura de fondo; y otras aproximaciones sustituyen el almacén de fragmentos de texto por una base de conocimiento estructurada, como un grafo de conocimiento, dando lugar a GraphRAG [3] (véanse las Secciones 2.4 y 2.5).

En este trabajo se implementan dos sistemas de recuperación sobre el mismo corpus, no uno solo: un RAG vectorial de tipo *Advanced RAG* –con recuperación híbrida y reordenación de resultados– y un sistema GraphRAG completo, con el fin de comparar ambos enfoques sobre las mismas preguntas (Capítulo 8).

2.4. Grafos de Conocimiento

Un grafo de conocimiento representa la información como una red de entidades (nodos) y relaciones entre ellas (aristas), en lugar de como texto libre [6]. El estándar utilizado en este trabajo es RDF (*Resource Description Framework*) [7], que modela cada hecho como una tripleta *sujeto-predicado-objeto* (por ejemplo, **proposición X** --presentadaPor-->Grupo PP). Sobre RDF se puede definir una ontología con OWL (*Web Ontology Language*) [8] –las clases y relaciones válidas del dominio– y consultar el grafo con el lenguaje SPARQL [9].

Una ventaja de este modelo frente al texto libre es el razonamiento automático: un motor de inferencia (*reasoner*) puede derivar hechos nuevos que no están escritos explícitamente pero se siguen de la ontología –por ejemplo, que una proposición sobre “desahucios” trata también, por herencia temática, de “vivienda”– sin que haga falta repetir esa relación en cada dato.

2.5. GraphRAG

GraphRAG es la variante de RAG en la que la fuente de recuperación no es un almacén de fragmentos de texto sino un grafo de conocimiento consultado mediante un lenguaje de consulta estructurado (SPARQL en este trabajo). En lugar de recuperar pasajes de texto parecidos a la pregunta, un LLM traduce la pregunta en lenguaje natural a una consulta formal, el motor de grafos la ejecuta y devuelve filas de datos exactos, que un segundo LLM convierte en una respuesta narrada.

Esto resulta especialmente adecuado para preguntas de agregación y razonamiento multi-salto –“¿qué grupo ha presentado más proposiciones sobre vivienda?”, “¿cuántas se han aprobado por año?”– que un sistema de recuperación de texto libre no puede responder con precisión, porque exigen contar y cruzar datos de todo el corpus, no solo encontrar un pasaje parecido a la pregunta. Es, junto con el RAG vectorial tradicional, uno de los dos sistemas comparados en este TFG.

2.6. El Pleno del Ayuntamiento de Bilbao

Este trabajo se ha desarrollado sobre la documentación pública del Pleno del Ayuntamiento de Bilbao, el órgano superior de gobierno y control del municipio, integrado por la alcaldía y los concejales electos en cada mandato. En sus sesiones, ordinarias o extraordinarias, los distintos grupos políticos y la propia alcaldía presentan proposiciones que se debaten y se someten a votación; el resultado de esa votación –aprobada, rechazada, aprobada con enmienda o decaída– queda recogido, junto con el resto del debate, en el acta de la sesión.

El Ayuntamiento publica estas actas, junto con el resto de documentación de cada



Figura 3: Fachada del Ayuntamiento de Bilbao
Fuente: Wikimedia Commons, licencia CC0 (dominio público).

sesión (orden del día, extractos, resumen), en su portal de actas del Pleno, de acceso público. Es precisamente este archivo –de casi veinte años y 3.422 proposiciones– el que constituye el corpus documental de este trabajo; su descripción detallada, junto con el proceso que lo transforma en datos estructurados, se recoge en el Capítulo 7.

3. Descripción de requerimientos y análisis del estado del arte

El corpus documental de partida –las actas del Pleno– se describe en detalle, junto con su preprocesado, en el Capítulo 7.

3.1. Requisitos del sistema

A partir de los objetivos específicos fijados en la Sección 1.2.1 se derivan los requisitos funcionales y no funcionales que debe cumplir el sistema.

3.1.1. Requisitos funcionales

ID	Descripción
RF1	Extraer de forma automatizada las proposiciones individuales de cada acta en PDF, junto con fecha, grupo o particular proponente, tema, entidades mencionadas, resultado de la votación y página de origen.
RF2	Construir un grafo de conocimiento RDF/OWL que modele proposiciones, grupos políticos, concejales y temas, con razonamiento automático sobre relaciones temáticas jerárquicas (<i>roll-up</i> vía SKOS).
RF3	Derivar el resultado de la votación y el voto nominal por concejal mediante una capa determinista basada en el texto literal del acta, que prevalezca sobre la clasificación del LLM cuando el acta permite verificarla.
RF4	Implementar un sistema de RAG vectorial con recuperación híbrida (semántica, literal y temática) y reordenación de resultados.
RF5	Implementar un sistema GraphRAG que traduzca la pregunta del usuario a una consulta SPARQL, la ejecute sobre el grafo y narre la respuesta a partir de los datos obtenidos.
RF6	Citar, para cada dato mostrado por cualquiera de los dos sistemas, el documento y la página exacta del PDF de origen.
RF7	Exponer ambos sistemas mediante una interfaz web conversacional común.
RF8	Validar la integridad estructural del grafo de conocimiento mediante restricciones SHACL, de forma independiente del código que lo construye.
RF9	Evaluar de forma comparativa ambos sistemas sobre un conjunto de preguntas de referencia con respuesta verificada a mano contra las actas originales.

Tabla 1: Requisitos funcionales del sistema

3.1.2. Requisitos no funcionales

ID	Descripción
RNF1	Trazabilidad. Toda respuesta debe poder contrastarse contra el documento original; no es aceptable una respuesta sin fuente citable.
RNF2	Precisión sobre cobertura. Ante una pregunta que el sistema no pueda responder con datos verificables, se prefiere que lo indique explícitamente antes que generar una respuesta plausible pero no respaldada por el corpus.
RNF3	Coste. El sistema debe poder operar con modelos de lenguaje locales (Ollama) o con niveles gratuitos de API en la nube, sin depender de un presupuesto de inferencia elevado.
RNF4	Idioma. Las consultas y respuestas se producen en castellano, aunque el corpus contiene fragmentos en euskera.
RNF5	Mantenibilidad. La capa determinista y la capa basada en LLM deben mantenerse separadas, de forma que un error sistemático de clasificación pueda corregirse sin reenriquecer ni reentrenar.

Tabla 2: Requisitos no funcionales del sistema

3.2. Estado del arte de las tecnologías utilizadas

Además de los conceptos generales de LLM, RAG y GraphRAG presentados en el Capítulo 2, este trabajo se apoya en dos líneas de investigación más específicas: la mejora de la recuperación en sistemas RAG mediante reordenación de resultados, y la fiabilidad de la generación de consultas formales (SPARQL o SQL) a partir de lenguaje natural con LLM –el componente más crítico del sistema GraphRAG, porque un error aquí produce una cifra incorrecta narrada como si fuera un hecho verificado, en vez de un fallo visible.

3.2.1. Recuperación híbrida y reordenación de resultados

La combinación de varios canales de recuperación (semántico, léxico y, en este caso, temático) con una reordenación posterior de los resultados es una de las mejoras características de lo que la literatura denomina *Advanced RAG* frente al esquema *Naive RAG* [10]. La reordenación mediante un modelo *cross-encoder* –que evalúa conjuntamente la pregunta y cada fragmento candidato, en vez de compararlos por separado como hace la búsqueda por *embeddings*– fue propuesta originalmente para recuperación de pasajes con BERT [11] y hoy es una técnica estándar para mejorar la precisión de los primeros resultados devueltos por un motor de recuperación semántica. En este trabajo se aplica mediante el servicio de reordenación de Cohere sobre los candidatos de los tres canales de búsqueda (Capítulo 6).

3.2.2. Fiabilidad de la generación de SPARQL con LLM

La traducción de lenguaje natural a consultas formales mediante LLM es un área de investigación activa, tanto para SPARQL sobre grafos de conocimiento como para SQL sobre bases de datos relacionales –problema estructuralmente equivalente al que resuelve el sistema GraphRAG de este trabajo. La literatura converge en varios hallazgos que han

condicionado directamente las decisiones de diseño de este sistema (desarrolladas con más detalle en los Capítulos 5 y 6):

- **El *zero-shot* puro es insuficiente.** Sin ejemplos de referencia en el *prompt*, un LLM genera consultas SPARQL sintácticamente válidas mucho más a menudo que semánticamente correctas; incluir ejemplos (*few-shot*) mejora sustancialmente la tasa de acierto [12].
- **La selección de ejemplos por similitud no es automáticamente superior a alternativas más simples.** Un estudio sobre un grafo de conocimiento de accidentes de aviación encontró que un *prompt* simple con ejemplos aleatorios podía igualar a estrategias de selección por similitud semántica [13], mientras que en *text-to-SQL* la evaluación comparativa de Gao et al. sí encuentra que elegir los ejemplos por similitud supera a elegirlos al azar en todos los modelos probados [14]; la ganancia depende en buena medida de la capacidad del modelo base para aprovechar esa selección.
- **El tamaño del modelo tiene rendimientos decrecientes.** En la generación de consultas, la técnica de *prompting* importa tanto o más que el número de parámetros: en la evaluación comparativa de Gao et al., un modelo anterior con ejemplos bien elegidos supera a GPT-4 sin ejemplos [14]. Además, modelos pequeños (de 1 000 a 15 000 millones de parámetros) afinados a la tarea pueden superar a modelos mucho mayores sin ese afinamiento [15].
- **Los sistemas basados en plantillas fijas siguen siendo una alternativa con precedente** para preguntas dentro de un catálogo conocido, a cambio de perder flexibilidad ante formulaciones no previstas –el compromiso clásico entre cobertura y fiabilidad, presente desde los primeros sistemas de pregunta-respuesta sobre RDF [16].

Estos hallazgos –en particular, la insuficiencia del *zero-shot*, la utilidad limitada de la selección por similitud según el modelo, y los rendimientos decrecientes del tamaño del modelo– se contrastan con resultados propios obtenidos sobre el grafo de este proyecto en el Capítulo 5, y motivan directamente la capa determinista de corrección de consultas SPARQL descrita en el Capítulo 6.

4. Planificación del proyecto

4.1. Diagrama EDT

La Figura 4 muestra la Estructura de Desglose del Trabajo (EDT) del proyecto, con los cinco paquetes de trabajo de primer nivel y sus subpaquetes, que se detallan en la Sección 4.2.

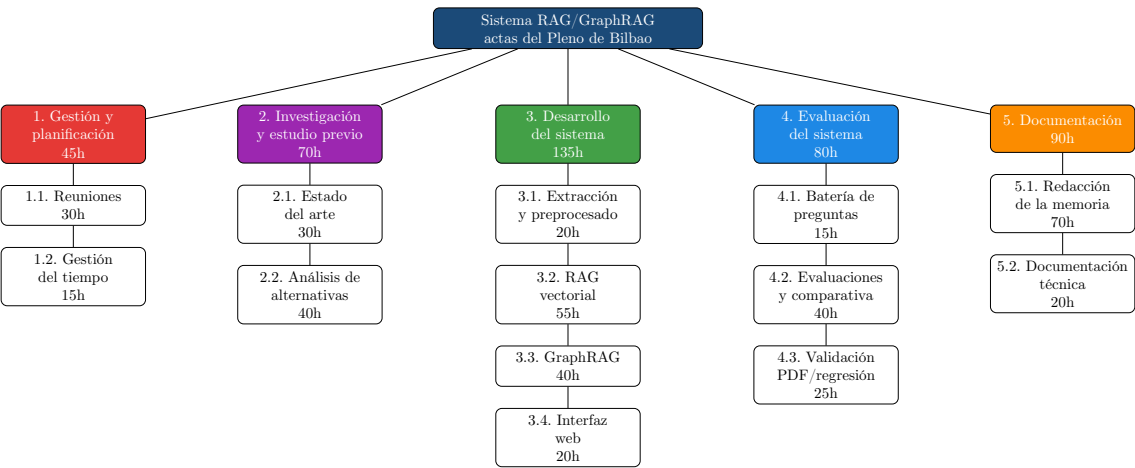


Figura 4: Diagrama EDT del proyecto

4.2. Descripción de tareas

En esta sección se describen los paquetes de trabajo mostrados en el diagrama EDT. Cada paquete se presenta en una tabla con su tiempo estimado, descripción, recursos empleados, predecesores y entregables –mismo formato en toda la memoria, un color por categoría de primer nivel del EDT.

1. Gestión y planificación	
Tiempo estimado	45 horas
Descripción	Tareas relacionadas con la planificación del proyecto y la gestión del tiempo con el director.
Recursos	Reuniones periódicas con el director, herramienta de diagramación.
Predecesores	-
Entregables/salidas	Diagramas EDT y Gantt, apuntes de seguimiento del trabajo.

Tabla 3: Paquete de trabajo: 1. Gestión y planificación

1.1. Reuniones con el director	
Tiempo estimado	30 horas
Descripción	Reuniones periódicas de seguimiento con el director para revisar el avance, resolver dudas técnicas y ajustar la planificación.
Recursos	Videollamada / presencial.
Predecesores	-
Entregables/salidas	Valoraciones del trabajo realizado y tareas siguientes acordadas.

Tabla 4: Paquete de trabajo: 1.1. Reuniones con el director

1.2. Gestión del tiempo	
Tiempo estimado	15 horas
Descripción	Planificación y seguimiento del tiempo dedicado a cada paquete de trabajo; elaboración de los diagramas EDT y Gantt.
Recursos	Herramienta de diagramación (Draw.io).
Predecesores	-
Entregables/salidas	Diagramas EDT y Gantt.

Tabla 5: Paquete de trabajo: 1.2. Gestión del tiempo

2. Investigación y estudio previo	
Tiempo estimado	70 horas
Descripción	Estudio de las técnicas y tecnologías en las que se apoya el sistema, y análisis de las principales alternativas tecnológicas antes de fijar la arquitectura.
Recursos	Los recursos empleados se especifican en cada subpaquete.
Predecesores	-
Entregables/salidas	Capítulos de Contexto y Análisis de alternativas de la memoria.

Tabla 6: Paquete de trabajo: 2. Investigación y estudio previo

2.1. Estudio del estado del arte	
Tiempo estimado	30 horas
Descripción	Estudio de LLM, RAG y sistemas GraphRAG existentes, y de trabajos similares sobre documentación administrativa.
Recursos	Documentación técnica y bibliografía de referencia.
Predecesores	-
Entregables/salidas	Capítulo de Contexto y estado del arte de la memoria.

Tabla 7: Paquete de trabajo: 2.1. Estudio del estado del arte

2.2. Análisis de alternativas tecnológicas	
Tiempo estimado	40 horas
Descripción	Comparación de LLM local frente a API en la nube, almacenamiento del grafo (rdflib/OWL-RL frente a una base de grafos dedicada) y estrategia de recuperación vectorial, incluyendo el diseño de la ontología OWL y el enfoque de razonamiento de GraphRAG.
Recursos	Documentación de las tecnologías evaluadas, pruebas exploratorias.
Predecesores	2.1. Estudio del estado del arte
Entregables/salidas	Capítulo de Análisis de alternativas de la memoria.

Tabla 8: Paquete de trabajo: 2.2. Análisis de alternativas tecnológicas

3. Desarrollo del sistema	
Tiempo estimado	135 horas
Descripción	Implementación del corpus estructurado, los dos sistemas de recuperación de información y la interfaz web.
Recursos	Los recursos empleados se especifican en cada subpaquete.
Predecesores	2.2. Análisis de alternativas tecnológicas
Entregables/salidas	Sistema completo: corpus estructurado, grafo de conocimiento, RAG vectorial, GraphRAG e interfaz web.

Tabla 9: Paquete de trabajo: 3. Desarrollo del sistema

3.1. Extracción y preprocesado del corpus	
Tiempo estimado	20 horas de trabajo activo
Descripción	Scraping del portal de actas, extracción de proposiciones y enriquecimiento temático mediante LLM (Capítulo 7). No incluye el tiempo de cómputo del propio enriquecimiento, que corre sin supervisión (ver Sección 4.5).
Recursos	Python, LangChain, API de Gemini.
Predecesores	2.2. Análisis de alternativas tecnológicas
Entregables/salidas	Corpus estructurado de proposiciones (<code>proposals_enriched.jsonl</code>).

Tabla 10: Paquete de trabajo: 3.1. Extracción y preprocesado del corpus

3.2. RAG vectorial	
Tiempo estimado	55 horas
Descripción	Implementación de la recuperación híbrida (semántica, literal y temática), reordenación de resultados y generación de respuestas con citación de fuentes.
Recursos	ChromaDB, LangChain, Cohere (reordenación), Groq/Ollama (generación).
Predecesores	3.1. Extracción y preprocesado del corpus
Entregables/salidas	Sistema RAG vectorial funcional.

Tabla 11: Paquete de trabajo: 3.2. RAG vectorial

3.3. GraphRAG	
Tiempo estimado	40 horas
Descripción	Construcción del grafo RDF/OWL, razonamiento automático, generación de SPARQL a partir de lenguaje natural y sanitización de los anti-patrones más frecuentes en las consultas generadas por el LLM.
Recursos	rdflib, owlrl, Ollama (generación de SPARQL y narración).
Predecesores	3.1. Extracción y preprocesado del corpus
Entregables/salidas	Sistema GraphRAG funcional (<code>bilbao_reasoned.ttl</code> + <code>graph_rag_sparql.py</code>).

Tabla 12: Paquete de trabajo: 3.3. GraphRAG

3.4. Interfaz web	
Tiempo estimado	20 horas
Descripción	Integración de ambos sistemas de recuperación en una interfaz web conversacional común, con citación de fuentes enlazada a la página exacta del PDF. Al construirse sobre Chainlit, un <i>framework</i> ya dado, el esfuerzo se centra en la integración de los dos <i>backends</i> , no en construir la interfaz desde cero.
Recursos	Chainlit.
Predecesores	3.2. RAG vectorial, 3.3. GraphRAG
Entregables/salidas	Interfaz web funcional.

Tabla 13: Paquete de trabajo: 3.4. Interfaz web

4. Evaluación del sistema	
Tiempo estimado	80 horas
Descripción	Diseño y ejecución de la evaluación comparativa de ambos sistemas, y validación de las respuestas contra las actas originales en PDF.
Recursos	Los recursos empleados se especifican en cada subpaquete.
Predecesores	3.4. Interfaz web
Entregables/salidas	Resultados de la evaluación comparativa (Capítulo 8).

Tabla 14: Paquete de trabajo: 4. Evaluación del sistema

4.1. Diseño de la batería de preguntas de referencia	
Tiempo estimado	15 horas
Descripción	Diseño de las preguntas de referencia y verificación manual de su respuesta correcta contra las actas originales.
Recursos	Lectura directa de las actas en PDF.
Predecesores	3.4. Interfaz web
Entregables/salidas	Batería de preguntas de referencia con respuesta verificada.

Tabla 15: Paquete de trabajo: 4.1. Diseño de la batería de preguntas de referencia

4.2. Evaluaciones y comparativa	
Tiempo estimado	40 horas
Descripción	Ejecución de la batería sobre ambos sistemas, análisis de la calidad de recuperación y de generación, y comparativa GraphRAG frente a RAG vectorial.
Recursos	Scripts de evaluación propios.
Predecesores	4.1. Diseño de la batería de preguntas de referencia
Entregables/salidas	Resultados comparativos entre ambos sistemas.

Tabla 16: Paquete de trabajo: 4.2. Evaluaciones y comparativa

4.3. Validación contra PDF real y regresión	
Tiempo estimado	25 horas
Descripción	Verificación manual del contenido de las respuestas contra el texto original de las actas (no solo contra la estructura de la respuesta), y ejecución de la suite de regresión ante cada cambio relevante.
Recursos	scripts/regression_qa.py, lectura directa de PDF.
Predecesores	4.2. Evaluaciones y comparativa
Entregables/salidas	Conjunto de casos verificados y corregidos (véase decisiones_tecnicas.md para el detalle de los casos encontrados).

Tabla 17: Paquete de trabajo: 4.3. Validación contra PDF real y regresión

4.2.1. Gestión del proyecto

4.2.2. Investigación y estudio previo

4.2.3. Desarrollo del sistema

4.2.4. Evaluación del sistema

4.2.5. Documentación

5. Documentación	
Tiempo estimado	90 horas
Descripción	Redacción de la memoria del TFG y de la documentación técnica del proyecto.
Recursos	Los recursos empleados se especifican en cada subpaquete.
Predecesores	4.3. Validación contra PDF real y regresión
Entregables/salidas	Memoria del TFG.

Tabla 18: Paquete de trabajo: 5. Documentación

5.1. Redacción de la memoria	
Tiempo estimado	70 horas
Descripción	Redacción de los capítulos de la memoria del TFG.
Recursos	L ^A T _E X (plantilla EHU).
Predecesores	4.3. Validación contra PDF real y regresión
Entregables/salidas	Memoria del TFG.

Tabla 19: Paquete de trabajo: 5.1. Redacción de la memoria

5.2. Documentación técnica	
Tiempo estimado	20 horas
Descripción	Documentación del razonamiento detrás de las decisiones técnicas no evidentes del sistema, separada del código (<code>decisiones_tecnicas.md</code>), y de los procedimientos de despliegue y uso (<i>READMEs</i> , <i>runbook</i>).
Recursos	Markdown.
Predecesores	- (paralelo al resto de paquetes)
Entregables/salidas	<code>decisiones_tecnicas.md</code> , <i>READMEs</i> y <i>runbook</i> del proyecto.

Tabla 20: Paquete de trabajo: 5.2. Documentación técnica

4.3. Gestión del tiempo

4.3.1. Estimación de horas frente a duración real

4.3.2. Diagrama de Gantt

La Figura 5 muestra la planificación temporal del proyecto, desde el inicio de los primeros trabajos (22 de enero de 2026) hasta la fecha límite de entrega de la memoria (15 de octubre de 2026), repartiendo las 420 horas estimadas de la EDT (Sección 4.2) en 38 semanas. Las tareas de gestión (1.1 y 1.2) y la documentación técnica (5.2) se desarrollan de forma continua en paralelo al resto, en vez de en un bloque cerrado.

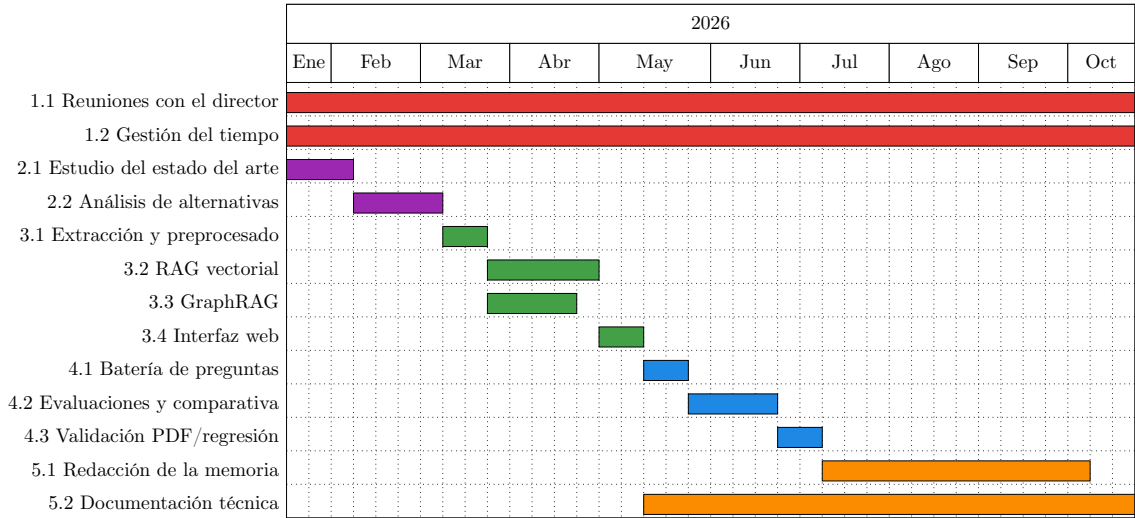


Figura 5: Diagrama de Gantt del proyecto (semanas desde el 22 de enero de 2026)

4.4. Metodología de desarrollo

El desarrollo ha seguido un enfoque iterativo: cada componente se implementa, se prueba en vivo contra preguntas reales y se corrige antes de pasar al siguiente, en vez de implementar el sistema completo y evaluarlo solo al final. Cada iteración de corrección se apoya en dos verificaciones complementarias: una suite de regresión automática (`scripts/regression_qa.py`) que se relanza tras cualquier cambio relevante en la recuperación o la generación de SPARQL, y una validación manual contra el texto original de las actas en PDF –no solo contra la estructura de la respuesta o el propio sistema– para detectar fallos que la regresión automática no puede ver, como una mezcla de contenido entre dos proposiciones distintas dentro de un mismo bloque bien formado. Este proceso, con sus hallazgos y las correcciones aplicadas en cada ronda, queda documentado en `decisiones_tecnicas.md`, que sirve además de base directa para el Capítulo 6 de esta memoria.

4.5. Evaluación económica

4.5.1. Costes por mano de obra

El coste de mano de obra se calcula a partir del total de horas estimadas en el diagrama EDT (Sección 4.2), valoradas a la tarifa de un ingeniero junior. En vez de una cifra de referencia arbitraria, se toma el salario mínimo real fijado por convenio: el XIX Convenio colectivo estatal de empresas de consultoría, tecnologías de la información y estudios de mercado y de la opinión pública [17] establece una jornada anual de 1.800 horas (art. 20), y su tabla salarial actualizada para 2026 [18] fija en 29.949,83 €/año el salario de la categoría “Titulado de grado superior” –la que corresponde a un Grado universitario de 240 créditos ECTS como el de este TFG, no la categoría inferior “Titulado de grado medio” (antigua diplomatura de 3 años)–. Esto equivale a una tarifa de 16,64 €/h.

Concepto	Valor
Horas totales estimadas	420 h
Tarifa/hora (29.949,83 €/año “Titulado de grado superior” ÷ 1.800 h/año)	16,64 €/h
Coste de mano de obra	6.988,29 €

Tabla 21: Coste de mano de obra

4.5.2. Costes en hardware

El proyecto se ha desarrollado con dos equipos propios: un portátil, usado para la práctica totalidad del trabajo activo (desarrollo, redacción, reuniones), y un ordenador de sobremesa, usado únicamente para el cómputo sin supervisión que no requiere intervención activa –el enriquecimiento LLM local del corpus completo y el estudio de comparación de modelos para la generación de SPARQL (Sección 3.2.2)–. Ambos equipos se amortizan de forma proporcional a las horas dedicadas al proyecto sobre una vida útil estimada de 4 años (~7.000 horas de uso).

¹Precio y horas de uso estimados. Especificaciones del ordenador de sobremesa: AMD Ryzen 5 3600 (6 núcleos / 12 hilos), 16 GB de RAM, GPU NVIDIA GeForce RTX 2060 con 6 GB de VRAM, Windows 10 Pro de 64 bits.

Equipo	Precio	Horas usadas	Vida útil	Coste
Portátil (Huawei, Ryzen 5)	650 €	420 h	7.000 h	39 €
Ordenador de sobremesa ¹	900 €	65 h	7.000 h	8 €
Coste de hardware				47 €

Tabla 22: Coste de hardware (amortizado)

4.5.3. Costes en software

Todas las herramientas de software empleadas son de código abierto o se han usado en su capa gratuita (ChromaDB, LangChain, Chainlit, Ollama, Groq y el plan de prueba de Cohere), salvo el enriquecimiento temático del corpus con la API de pago de Gemini, cuyo coste real quedó medido y documentado durante el desarrollo.

Servicio	Coste
Groq (capa gratuita)	0 €
Cohere (plan de prueba)	0 €
Ollama, ChromaDB, LangChain, Chainlit (código abierto)	0 €
API de Gemini (enriquecimiento del corpus)	7 €
Coste de software	7 €

Tabla 23: Coste de software

4.5.4. Costes indirectos

Se considera como coste indirecto el consumo eléctrico de ambos equipos durante las horas dedicadas al proyecto, a una tarifa media residencial de referencia de 0,20 €/kWh.

Equipo	Consumo	Horas	Coste
Portátil	~50 W	420 h	4 €
Ordenador de sobremesa	~300 W	65 h	4 €
Coste indirecto (electricidad)			8 €

Tabla 24: Costes indirectos (electricidad)

4.5.5. Coste total

El coste de mano de obra domina claramente el presupuesto (~99 % del total), lo cual es coherente con un proyecto que no requiere hardware especializado ni servicios de pago más allá de una única pasada puntual de enriquecimiento del corpus.

4.6. Análisis de riesgos

4.6.1. Medidas de prevención y contingencia

Concepto	Coste
Mano de obra	6.988,29 €
Hardware (amortizado)	47 €
Software	7 €
Indirectos (electricidad)	8 €
Coste total	7.050,29 €

Tabla 25: Coste total del proyecto

1. Dependencia de APIs externas (Groq)	
Descripción	El sistema usa la API de Groq para generar las respuestas finales; si el servicio falla o se agota la cuota diaria, la calidad de la respuesta puede degradarse.
Plan de prevención	Vigilar el consumo de cuota antes de lanzar baterías de pruebas extensas.
Plan de contingencia	<i>Fallback</i> automático a un modelo local vía Ollama cuando Groq no responde o se agota la cuota.
Probabilidad	Media
Impacto	Bajo (el sistema sigue funcionando, con menor calidad de redacción)

Tabla 26: Riesgo: 1. Dependencia de APIs externas

2. Errores silenciosos en la generación de SPARQL	
Descripción	El LLM puede generar una consulta SPARQL sintácticamente válida pero semánticamente incorrecta (patrón que cuenta de más, propiedad inventada, filtro perdido...); el resultado es una cifra equivocada narrada como si fuera un hecho verificado, sin ningún error visible que la delate.
Plan de prevención	Catálogo de anti-patrones detectados y sanitizados automáticamente antes de ejecutar la consulta (<i>decisiones_tecnicas.md</i> , Sección 3.1), y validación manual contra el PDF original –no solo contra la estructura de la respuesta– en cada ronda de pruebas.
Plan de contingencia	Suite de regresión automática (<i>scripts/regression_qa.py</i>) que se relanza tras cualquier cambio en la generación de SPARQL, para detectar si una corrección nueva reintroduce un fallo ya solucionado.
Probabilidad	Alta (es, de hecho, el fallo más frecuente encontrado durante el desarrollo).
Impacto	Alto: una cifra incorrecta sin error visible es el peor caso posible para un sistema cuyo requisito central es la trazabilidad (RNF1, Capítulo 3).

Tabla 27: Riesgo: 2. Errores silenciosos en la generación de SPARQL

3. Calidad variable del OCR en actas antiguas	
Descripción	Las actas más antiguas del corpus (proceden de PDF escaneados o convertidos de forma menos cuidada) presentan errores de OCR –espacios sueltos, mezcla de euskera y castellano en los mismos nombres, cortes de palabra– que dificultan la extracción de proposiciones y la identificación de concejales.
Plan de prevención	Deduplicación de nombres de concejal mediante plegado fonético de apellidos, solape de nombres de pila y un mapa explícito de alias euskera/castellano (Sección 5.2 de <code>decisiones_tecnicas.md</code>).
Plan de contingencia	Cuando la resolución automática no alcanza confianza suficiente, la entidad se deja sin resolver en vez de asignarla de forma incorrecta –se prioriza no introducir un dato falso sobre completar el 100 % de los casos.
Probabilidad	Media (concentrada en las actas más antiguas del corpus).
Impacto	Medio: afecta a la cobertura del voto nominal y a la atribución de partido de algunos concejales, no a la fiabilidad de los datos que sí se resuelven.

Tabla 28: Riesgo: 3. Calidad variable del OCR en actas antiguas

4. Plazo de entrega ajustado con un único desarrollador	
Descripción	El proyecto lo desarrolla una sola persona, con fecha límite de entrega de la memoria el 15 de octubre de 2026 (Sección 4.2); cualquier imprevisto –de salud, técnico o de disponibilidad– no tiene margen de reasignación a otro miembro del equipo.
Plan de prevención	Seguimiento semanal del avance frente al diagrama de Gantt (Figura 5), y priorización explícita de los requisitos funcionales (Tabla 1) sobre mejoras no esenciales si el margen se estrecha.
Plan de contingencia	Recorte de alcance dentro de lo ya delimitado en la Sección 1.2.1 –p. ej. reducir el tamaño de la batería de preguntas de referencia– antes que sacrificar la trazabilidad o la corrección de los datos mostrados.
Probabilidad	Media.
Impacto	Alto: sin margen de reasignación, un imprevisto afecta directamente a la fecha de entrega.

Tabla 29: Riesgo: 4. Plazo de entrega ajustado con un único desarrollador

5. Nombres y grupos ambiguos o mal atribuidos en las actas	
Descripción	Las actas identifican a los concejales por apellido suelto, y varios comparten apellido dentro del censo (p. ej. tres concejales distintos apellidados “Gil” en mandatos distintos); sin desambiguar, el voto nominal puede atribuirse a la persona equivocada. De forma similar, una palabra genérica puede confundirse con el nombre de un grupo político (“ciudadanos” en minúscula generaba falsos positivos del partido Ciudadanos antes de corregirse).
Plan de prevención	Resolución de apellido por (apellido + mandato) contra el censo de concejales de esa legislatura, en vez de por apellido suelto; comparación exacta de grupo político contra los alias canónicos del partido en vez de coincidencia de texto libre.
Plan de contingencia	Cuando varios concejales del censo comparten apellido y grupo en la misma fecha, se emite únicamente el voto agregado por grupo (bo:votoAFavorDe) y se omite el voto nominal de esa persona, en vez de asignarlo al primer candidato activo y arriesgarse a acertar el grupo pero fallar la persona.
Probabilidad	Media (concentrada en apellidos comunes con varios concejales activos en mandatos distintos).
Impacto	Medio: afecta a la atribución de voto nominal a personas concretas, no al recuento agregado por grupo, que es el dato que más preguntas de agregación utilizan.

Tabla 30: Riesgo: 5. Nombres y grupos ambiguos o mal atribuidos en las actas

5. Análisis de alternativas

5.1. Modelos de lenguaje: local (Ollama) frente a API en la nube (Groq / Gemini)

El sistema necesita LLM en dos contextos muy distintos: generación **interactiva** (una respuesta por pregunta del usuario, en tiempo real) y enriquecimiento **por lotes** (clasificar las 3.422 proposiciones del corpus una sola vez, sin supervisión). Se evaluaron tres opciones para ambos: modelos locales servidos con Ollama, la API gratuita de Groq, y la API de pago de Gemini.

5.1.1. Puntos positivos

- **Ollama (local):** sin coste por token ni límite de cuota; los datos no salen de la máquina, relevante al tratarse de un corpus público pero cuyo procesamiento no depende de un tercero.
- **Groq:** nivel gratuito con inferencia muy rápida (`gpt-oss-120b` sirve respuestas interactivas sin necesidad de GPU propia); se usa como primera opción para la generación interactiva del RAG vectorial, con caída automática a Ollama local si falla (Sección 2.1 de `decisiones_tecnicas.md`).
- **Gemini:** coste bajo y predecible con el modelo `gemini-3.5-flash-lite` (0,30 \$/1,50 \$ por millón de tokens entrada/salida); el enriquecimiento completo del corpus (~3.400 proposiciones) costó ~5 € medidos en producción.

5.1.2. Puntos negativos

- **Ollama (local):** sin GPU dedicada, la generación es inviable para procesos por lotes: ~200 s por proposición en un portátil de 6 GB de RAM habría supuesto más de 180 horas para el corpus completo. Requiere un equipo con GPU reservado para esa tarea (Sección 4.5).
- **Groq:** el nivel gratuito limita a ~100-200k tokens/día, cuota asociada a la clave de API –no se evita usando otra máquina–, insuficiente para enriquecer el corpus completo de una sola vez.
- **Gemini:** de pago desde el primer token; requiere control activo del gasto (`--max-eur` en `build_graph.py`) y los modelos disponibles para una misma clave de API cambian con el tiempo (`gemini-2.5-flash` y `2.5-flash-lite` dejaron de estar disponibles durante el desarrollo).

5.1.3. Conclusión

No se elige una única opción, sino una combinación según el contexto de uso: **Groq** para la generación interactiva (rápida y gratuita, con Ollama local como respaldo ante fallos de cuota), y **Gemini Flash-Lite** para el enriquecimiento por lotes del corpus completo (coste bajo y predecible, evita las ~180 horas que habría requerido Ollama sin GPU). Ollama local con GPU se reserva para el estudio comparativo de modelos de generación de SPARQL (Sección 3.2.2), donde repetir el mismo experimento decenas de veces sobre el mismo modelo hace prohibitivo cualquier coste por token. Que un modelo abierto ejecutado en local sea candidato razonable para generar consultas no es una apuesta a ciegas: la literatura sobre *text-to-SQL* muestra que modelos abiertos de tamaño moderado resultan competitivos en esta tarea, sobre todo cuando se combinan con buenas técnicas de *prompting* o se afinan a la tarea [14, 15].

5.2. Almacenamiento del grafo: rdflib/OWL-RL frente a una base de grafos dedicada (p.ej. GraphDB, Neo4j)

El grafo de conocimiento (~247.500 triples tras el razonamiento OWL-RL) puede almacenarse y consultarse con una librería en memoria (rdflib + owlrl) o con un motor de grafos dedicado desplegado como servicio (GraphDB, Neo4j, Blazegraph...).

5.2.1. Puntos positivos

- **rdflib/OWL-RL:** no requiere levantar ni mantener un servicio aparte –el grafo es un archivo (`bilbao_reasoned.ttl`) que se carga en memoria al arrancar–, lo que simplifica el despliegue y es coherente con el requisito de coste bajo (RNF3, Capítulo 3). El razonamiento OWL-RL se ejecuta una vez, de forma *offline*, al construir el grafo, no en cada consulta.
- **Base de grafos dedicada:** índices optimizados para consultas SPARQL sobre grafos de gran tamaño, soporte nativo de concurrencia entre usuarios, y herramientas de administración e inspección visual del grafo.

5.2.2. Puntos negativos

- **rdflib/OWL-RL:** el grafo completo se carga en memoria en cada arranque del proceso, y las consultas SPARQL complejas sobre grafos grandes son más lentas que con un motor dedicado con índices persistentes –a este volumen de triples, sin impacto perceptible en la interfaz conversacional, pero es una limitación que no escala indefinidamente.
- **Base de grafos dedicada:** añade una pieza de infraestructura más que instalar, configurar y mantener –y, en las opciones no libres, un coste de licencia o de alojamiento– para un proyecto de un único desarrollador y un grafo que cabe cómodamente en memoria.

5.2.3. Conclusión

Se descarta una base de grafos dedicada para esta entrega: el volumen del grafo (~247.500 triples) no lo justifica, y `rdflib`/OWL-RL cumple el requisito de trazabilidad y razonamiento automático (RF2, RF8) sin coste de infraestructura adicional. Si el corpus creciera en varios órdenes de magnitud, o si el sistema tuviera que atender consultas concurrentes de múltiples usuarios con baja latencia, un motor de grafos dedicado pasaría a ser la opción preferible.

5.3. Almacén vectorial y recuperación: ChromaDB e híbrida (semántica + literal + temática) frente a solo semántica

El RAG vectorial puede limitarse a la búsqueda semántica clásica por *embeddings*, o combinarla con canales adicionales de recuperación sobre el mismo almacén (ChromaDB).

5.3.1. Puntos positivos

- **Solo semántica:** arquitectura más simple, un único canal de recuperación y sin lógica adicional de fusión de resultados.
- **Híbrida (semántica + literal + temática):** cada canal cubre un fallo distinto de los otros dos. El canal literal recupera menciones únicas de nombres propios que el *embedding* no acerca lo suficiente (p. ej. una empresa citada una sola vez); el canal temático recupera fragmentos relevantes para preguntas generales sobre un tema amplio (“qué se ha hecho en movilidad”) que no comparten vocabulario textual con los fragmentos relevantes, porque mide coincidencia de *clasificación* en vez de coincidencia de *texto* (Sección 1.1 de `decisiones_tecnicas.md`).

5.3.2. Puntos negativos

- **Solo semántica:** pierde precisión exactamente en los dos casos que motivan los canales adicionales –menciones únicas y preguntas temáticas amplias–, con el riesgo de devolver “no hay información” cuando sí la hay en el corpus.
- **Híbrida:** más candidatos que fusionar y reordenar por pregunta (hasta tres canales, cada uno con sus propios candidatos), lo que exige una etapa de reordenación con un modelo *cross-encoder* [11] (Cohere) para que el volumen adicional de candidatos no degrade la relevancia de los primeros resultados, y añade complejidad de implementación y de depuración frente a un único canal.

5.3.3. Conclusión

Se adopta la recuperación híbrida sobre ChromaDB: el coste de implementación adicional se compensa con una cobertura de preguntas que la búsqueda semántica sola no alcanza, y es coherente con el requisito de precisión sobre cobertura (RNF2, Tabla 2) –preferir encontrar el dato correcto, aunque exija combinar varias estrategias de búsqueda, a que el sistema responda que no dispone de información cuando sí la tiene.

6. Descripción de la solución propuesta

El proceso de obtención y preparación del corpus sobre el que opera el sistema se describe aparte, en el Capítulo 7.

6.1. Arquitectura general del sistema

6.2. Construcción del grafo de conocimiento

6.3. Sistema de recuperación de información

6.3.1. RAG vectorial

6.3.2. GraphRAG

6.4. Generación de respuestas

6.5. Interfaz web del sistema

6.6. Flujo completo del sistema

6.7. Configuración y decisiones de implementación

6.7.1. Temperatura y parámetros de los LLM

6.7.2. Gestión de cuotas y memoria

6.7.3. Construcción del *prompt* para generación

7. Corpus y preprocesado de datos

7.1. El corpus de actas del Pleno

7.2. Pipeline de preprocesado

7.2.1. Descarga automatizada de las actas

7.2.2. Extracción de proposiciones

7.2.3. Enriquecimiento con un LLM

8. Análisis de resultados

8.1. Objetivo de la evaluación

Este TFG incluye dos sistemas de naturaleza distinta, y esa diferencia condiciona cómo se evalúa cada uno. El GraphRAG traduce la pregunta a SPARQL y devuelve datos exactos y verificables (conteos, rankings, ratios) sobre los que existe una única respuesta correcta calculable de antemano. El RAG vectorial, en cambio, responde con una crónica narrativa de contexto abierto para la que no hay una única redacción correcta contra la que comparar carácter a carácter. El objetivo de este capítulo es, por tanto, doble: verificar que el GraphRAG devuelve los datos correctos frente a una verdad conocida, y valorar si las respuestas del RAG vectorial son relevantes y están fundamentadas en las actas recuperadas, sin inventar información no presente en ellas.

8.2. Metodología

8.2.1. GraphRAG: precisión de ejecución

La calidad de la generación de SPARQL se evaluó en dos fases. Primero, un estudio comparativo entre 10 modelos de generación (Groq `gpt-oss-120b` y `gpt-oss-20b`, Gemini, y las variantes locales Gemma, Llama 3.2, Mistral, Qwen 2.5-7B, Qwen 2.5-Coder-7B, Qwen 2.5-14B y Qwen3:8B) sobre una batería de 35 preguntas de referencia con su consulta SPARQL de verdad (*gold*) y el valor esperado del resultado, clasificadas por tipo de razonamiento (conteo, ranking, ratio, temporal, persona, voto nominal, entidad, coautoría). Este estudio, ya descrito en la Sección 3.2.2, llevó a seleccionar Qwen3:8B ejecutado en local como generador de SPARQL de producción. Segundo, una suite de regresión continua (22 casos, archivo `scripts/regression_qa.py`) que no compara el texto narrado sino las filas devueltas por el SPARQL generado contra valores `expect` y `forbid` verificados manualmente contra las actas en PDF —es decir, **precisión de ejecución** (*execution accuracy*), el estándar habitual en la literatura de traducción de lenguaje natural a consultas formales (SQL o SPARQL) [12, 14]: más riguroso que un juez basado en LLM, porque compara contra un hecho verificado, no contra la opinión de otro modelo.

Esta suite no es una fotografía tomada al final del proyecto: se ha usado de forma continua durante todo el desarrollo para detectar regresiones cada vez que cambiaba el grafo o el generador de SPARQL, y varios de sus valores de referencia se establecieron verificando a mano el resultado contra el texto original de las actas. Ese proceso destapó fallos reales que una comparación automática contra el propio grafo nunca habría detectado: por ejemplo, un patrón de resultado (“decae la proposición”) ausente en la

expresión regular que clasifica el desenlace de cada proposición, que hacía que 82 de 3 422 proposiciones del grafo tuvieran un resultado mal clasificado; verificado el fallo contra el acta original, el arreglo redujo las discrepancias a 25.

8.2.2. RAG vectorial: RAGAS

Para el RAG vectorial no existe un único dato correcto que comparar, así que se recurrió a RAGAS [19], una librería de evaluación de sistemas RAG que usa un LLM externo como juez (*LLM-as-a-judge* [20]) sobre cuatro métricas, todas en el rango [0, 1]:

- **Context Relevance:** pertinencia temática de los fragmentos recuperados para la pregunta.
- **Faithfulness:** grado en que la respuesta generada se sostiene en el contexto recuperado, sin añadir información que no esté en él.
- **Answer Relevancy:** relevancia de la respuesta para la pregunta concreta que se hizo.
- **Semantic/Cosine Similarity:** similitud semántica entre la respuesta generada y una respuesta de referencia.

Las tres primeras corresponden a las tres puntuaciones de calidad –relevancia del contexto, fidelidad de la respuesta y relevancia de la respuesta– que la revisión de Gao et al. identifica como núcleo de la evaluación de sistemas RAG, y que cubren por separado las dos partes del sistema: la recuperación y la generación. La misma revisión recoge RAGAS entre las herramientas automáticas que calculan estas puntuaciones con un LLM [10].

Se construyó una batería de 50 preguntas de referencia. Las 13 primeras son las mismas que usa la suite de regresión para verificar la recuperación (`scripts/regression_qa.py`); las 37 restantes se eligieron a partir de los títulos de proposiciones reales del corpus, escogiendo a propósito temas con pocas proposiciones dedicadas (el Tour de Francia, la pobreza energética, los patinetes eléctricos, el Teatro Arriaga, el sector del taxi...), de modo que la respuesta de referencia pudiera verificarse *completa* y no solo en parte. La batería mezcla preguntas temáticas generales, preguntas acotadas a un grupo político y preguntas sobre un pleno concreto por su fecha. Cada pregunta tiene una respuesta de referencia escrita y verificada manualmente contra el texto de las actas en PDF –nunca generada a partir de lo que el propio sistema recuperase o respondiera, para no viciar la evaluación con su propio sesgo.

Como juez se usó `gemini-2.5-pro` a través de Vertex AI (Google Cloud). Antes se probó un juez local (Qwen3:8B), que se descartó por ser notablemente más indulgente: en una misma respuesta que atribuía mal una petición a un grupo, Qwen3 dio una Faithfulness de 1,00 y Gemini detectó el error y la bajó a 0,69. Es coherente con lo que observan Zheng et al.: la fiabilidad de un juez LLM depende de la capacidad del modelo que hace de juez, y un juez fuerte (GPT-4 en su estudio) alcanza con evaluadores humanos un nivel de acuerdo (en torno al 85 %) comparable al que hay entre las propias personas (en torno al 81 %) [20]. El embedding usado en las métricas que lo necesitan es `nomic-embed-text`, el mismo que usa el RAG vectorial en producción, para no mezclar dos nociones distintas de similitud semántica en la misma evaluación.

Las respuestas evaluadas se generaron en el equipo de sobremesa del proyecto (GPU NVIDIA RTX 2060 de 6 GB) con el generador local, Ollama Qwen2.5-7B-Instruct, y

no con Groq `gpt-oss-120b`. El sistema admite los dos generadores: Groq, gratuito a través de su API, permite usarlo en un portátil sin GPU, mientras que en un equipo con GPU puede funcionar íntegramente en local. El local es el escenario más exigente para la generación, por tratarse de un modelo mucho más pequeño, y el efecto de cambiar de generador se midió aparte con un experimento controlado (Sección 8.5). El resto del *pipeline* –recuperación híbrida y reordenamiento con Cohere– es el mismo en ambos casos. Los resultados completos, pregunta por pregunta, están en `scripts/ragas_results.csv`.

Una advertencia metodológica encontrada durante esta evaluación, por si resulta útil para trabajo futuro con RAGAS: las métricas de contexto deben construirse con el mismo texto que ve realmente el LLM generador –incluyendo metadatos como la fecha o el resultado de la votación que el sistema antepone a cada fragmento antes de generar la respuesta–, no solo el texto bruto del fragmento recuperado. Un primer intento que pasaba solo el texto bruto penalizaba Faithfulness artificialmente (0,08 en un caso verificado a mano) porque el juez no encontraba en el contexto una fecha que en realidad sí estaba disponible para el generador, solo que como metadato, no como texto plano; al corregirlo, la misma pregunta pasó a 0,81.

Conviene precisar qué relación hay entre las dos metodologías de este capítulo, porque no es la de “lo mismo, con y sin herramienta estandarizada”. RAGAS mide *consistencia*: si la respuesta es coherente con el contexto que el propio sistema dice haber recuperado, no si ese contexto –ni la respuesta– es correcto de verdad; un sistema puede obtener Faithfulness alta estando fundamentado en un contexto ya erróneo o incompleto, porque RAGAS nunca compara contra la fuente original. La verificación manual contra las actas en PDF y la precisión de ejecución del GraphRAG comparan, en cambio, contra la *verdad del documento o del grafo*, un chequeo más exigente que encontró fallos –como el de “decaecitado arriba– que una métrica de consistencia nunca habría detectado, porque no había ninguna inconsistencia entre la respuesta y el contexto recuperado: el fallo estaba antes, en el propio pipeline de extracción. RAGAS aporta lo que la verificación manual no puede dar –cobertura rápida y repetible, resumida en un número–, pero no la sustituye.

8.3. Resultados globales

La Tabla 31 recoge las cuatro métricas RAGAS sobre las 50 preguntas, en la configuración final del sistema. Las 50 se evaluaron completas, sin ningún fallo del juez.

Métrica	Media $\pm$ desviación típica ($n = 50$)
Context Relevance	0,99 $\pm$ 0,05
Faithfulness	0,76 $\pm$ 0,15
Answer Relevancy	0,65 $\pm$ 0,08
Semantic/Cosine Similarity	0,82 $\pm$ 0,04

Tabla 31: Métricas RAGAS del RAG vectorial en su configuración final (generador local Qwen2.5-7B)

Ninguna de las cuatro métricas cae en zona de alarma (por debajo de $\sim 0,5$, que indicaría un problema sistémico de recuperación o de generación), y ninguna pregunta obtiene un 0 en Context Relevance ni en Answer Relevancy. La lectura de la tabla es distinta según la métrica: las que dependen sobre todo de la recuperación (Context Relevance) están prácticamente en el máximo, mientras que las que dependen de cómo redacta el generador (Faithfulness y, en menor medida, Answer Relevancy) son las que tienen margen de mejora. Las dos secciones siguientes analizan cada parte por separado.

8.4. Análisis sobre la recuperación

Context Relevance obtiene una media de 0,99 con desviación típica 0,05: solo 2 de las 50 preguntas bajan de la puntuación máxima (ambas con 0,75). La recuperación híbrida de tres canales (semántico, literal por palabra clave y temático por la taxonomía SKOS del grafo, Sección 5.3) más el reordenamiento con Cohere trae fragmentos temáticamente pertinentes a la pregunta de forma consistente.

Este resultado no es el de la primera pasada de la evaluación, y la diferencia es uno de los hallazgos más útiles del capítulo. En una primera pasada, Context Relevance era de 0,95 con una desviación típica de 0,21: la media era alta, pero la hundían dos preguntas con un 0 absoluto –“¿Qué se ha debatido sobre los patinetes eléctricos y los vehículos de movilidad personal?” y “¿Qué iniciativas ha habido en defensa de los derechos del colectivo LGTBI?”–, en las que el sistema recuperaba contexto de otro tema (taxis y accesibilidad en la primera, violencia de género en la segunda) y respondía sobre él. Analizando estos dos casos se encontró que no fallaban los embeddings, sino la forma en que el canal literal elegía qué palabras buscar: se quedaba con las tres palabras más largas de la pregunta, de al menos seis letras. Ese criterio fallaba de tres maneras:

- **Siglas y palabras cortas descartadas.** “LGTBI” (cinco letras) nunca llegaba a buscarse, pese a aparecer en 283 fragmentos de 31 plenos distintos; en su lugar se buscaban “iniciativas”, “colectivo” y “derechos”, presentes en miles de fragmentos sin relación con la pregunta. En la batería, 21 de las 50 preguntas perdían alguna palabra con contenido por este motivo (OPE, taxi, Gaza, tala, Tour...).
- **Empates resueltos al azar.** “patinetes”, “vehículos” y “movilidad” tienen nueve letras; cuál de ellas entraba entre las tres elegidas dependía del orden interno de Python, que cambia en cada arranque del proceso. Se comprobó ejecutando la misma búsqueda en tres arranques: en uno “patinetes” quedó fuera (0 fragmentos relevantes) y en los otros dos entró (7). La misma pregunta podía, por tanto, funcionar o no según el arranque.
- **Palabras del enunciado ocupando puestos.** Términos como “debatido” (entre las tres elegidas en 12 de las 50 preguntas), “iniciativas” o “medidas” describen la pregunta, no el tema, y traían fragmentos al azar.

La longitud era un sustituto pobre de lo que realmente interesa: lo distintivo que es una palabra. La corrección pasó a elegir las tres palabras *más raras* en el corpus –contando en cuántos fragmentos aparece cada candidata y descartando las que no aparecen nunca–, con desempate determinista; a admitir palabras de cuatro o más letras y siglas en mayúsculas; y a excluir una lista de palabras propias del enunciado. La búsqueda literal pasó además a usar una expresión regular por palabra que ignora mayúsculas y tildes, cubre singular y plural y exige palabra completa (sin esto último, “tala” coincide dentro de “instalación”: más de 500 coincidencias frente a 72 reales). Como efecto colateral, el canal literal se volvió más rápido, de unos 6 s a entre 0,7 y 2,6 s por pregunta, porque sustituye una veintena de búsquedas (una por cada variante de escritura) por una sola consulta por palabra. La suite de regresión del RAG vectorial siguió pasando completa (13 de 13) tras el cambio.

La Tabla 32 compara las dos pasadas, con las mismas 50 preguntas, el mismo generador y el mismo juez.

La mejora se concentra donde se esperaba. Context Relevance y Answer Relevancy suben y, sobre todo, su desviación típica cae de forma marcada (de 0,21 a 0,05 en Context

Métrica ($n = 50$)	Antes de la corrección	Después
Context Relevance	$0,95 \pm 0,21$	$0,99 \pm 0,05$
Faithfulness	$0,76 \pm 0,19$	$0,76 \pm 0,15$
Answer Relevancy	$0,62 \pm 0,15$	$0,65 \pm 0,08$
Semantic/Cosine Similarity	$0,82 \pm 0,03$	$0,82 \pm 0,04$

Tabla 32: Métricas RAGAS antes y después de corregir la selección de palabras clave del canal literal

Relevance): desaparecen los fallos totales. La pregunta sobre el colectivo LGTBI pasa de 0 a 1 en Context Relevance y de 0 a 0,63 en Answer Relevancy; la de los patinetes, de 0 a 1 y de 0 a 0,59. Faithfulness, en cambio, no se mueve, lo cual es coherente: mide cómo redacta el generador a partir del contexto, y el generador no cambió. La lección metodológica es que la evaluación sobre una batería amplia encontró un fallo que la regresión dirigida no veía: los 13 casos de la suite pasaban todos con el criterio antiguo, porque ninguno dependía de una sigla ni de un empate.

8.5. Análisis sobre la generación

Faithfulness es la métrica más baja de las cuatro (0,76), y la pregunta obvia es si el problema está en la recuperación o en el generador. Para responderla se analizaron primero, afirmación por afirmación, las respuestas con menor puntuación: RAGAS descompone cada respuesta en afirmaciones sueltas y el juez indica, para cada una, si se sostiene en el contexto y por qué. Contrastando esos veredictos con el contexto recuperado, la mayoría de las afirmaciones marcadas como no fieles resultaron ser errores reales del generador, no fallos de medición, y se agrupan en tres patrones:

- **Leer “decae” como “aprobada”.** Es el más sistemático. En las actas es muy habitual el resultado “se aprueba la enmienda del Equipo de Gobierno, por lo que decae la proposición”: la proposición original *no* se aprueba, sino que la sustituye la enmienda. El generador se queda con “se aprueba” y presenta la proposición como aprobada, con los votos de la enmienda.
- **Mezclar información entre bloques.** Con varias proposiciones en el contexto, el generador atribuye a un pleno el título, el grupo o los votos de otro.
- **Inventar datos no presentes.** Por ejemplo, un orador (“el Sr. García”) que no aparece en ningún fragmento, o conclusiones genéricas que no se desprenden del contexto.

En la clasificación de Ji et al., los dos primeros son alucinaciones *intrínsecas* –la respuesta contradice el contexto del que parte– y el tercero es una alucinación *extrínseca* –añade información que el contexto no permite ni confirmar ni desmentir [5]. Los tres son errores de comprensión de un contexto ya recuperado correctamente, del mismo tipo que los documentados durante el desarrollo para la narración del GraphRAG con modelos de tamaño similar, y apuntan al generador y no a la recuperación. Para comprobarlo de forma directa se diseñó un experimento controlado: sobre un subconjunto de 22 preguntas –las 17 con Faithfulness inferior a 0,70 y 5 de control que ya habían obtenido una puntuación alta– se tomó *exactamente el mismo contexto* que había recibido el generador local y se volvió a redactar la respuesta con Groq `gpt-oss-120b`, con el mismo *prompt* y el mismo post-procesado. La única variable que cambia es el modelo que redacta. Las dos versiones se

puntuaron con el mismo juez y en la misma pasada. Las preguntas de control se incluyeron para descartar que una mejora en las peores se debiera solo a la variabilidad del juez (una segunda medición de un valor extremo tiende a acercarse a la media aunque nada haya cambiado). Este experimento se hizo antes de la corrección de la recuperación descrita en la Sección 8.4, con los contextos de la primera pasada; como compara generadores con el contexto fijo, esa diferencia no afecta a su conclusión.

Preguntas	Métrica	Qwen2.5-7B	gpt-oss-120b
Peores ($n = 17$)	Faithfulness	$0,62 \pm 0,16$	$0,79 \pm 0,14$
Control ($n = 5$)	Faithfulness	$0,70 \pm 0,36$	$0,99 \pm 0,03$
Todas ($n = 22$)	Faithfulness	$0,64 \pm 0,22$	$0,84 \pm 0,14$
Todas ($n = 22$)	Answer Rel.	$0,62 \pm 0,09$	$0,64 \pm 0,10$
Todas ($n = 22$)	Sem. Sim.	$0,81 \pm 0,03$	$0,82 \pm 0,03$

Tabla 33: Experimento controlado: mismo contexto recuperado, distinto generador

La Tabla 33 confirma la hipótesis. Con el mismo contexto, **gpt-oss-120b** obtiene mayor Faithfulness en 20 de las 22 preguntas, empata en una y queda por debajo solo en una; una prueba de signos sobre esos resultados descarta que la diferencia sea casual ($p < 0,001$). La mejora aparece tanto en las preguntas peor puntuadas como en las de control, lo que descarta que sea un simple efecto de volver a medir. En la práctica, Groq redacta correctamente el caso típico del patrón “decae”: “Aceptada la enmienda de modificación del Equipo de Gobierno, por lo que decae la proposición”. Answer Relevancy y Semantic Similarity apenas cambian, lo que también es coherente: dependen sobre todo de qué se recuperó, y eso era idéntico en las dos condiciones.

La conclusión es que la Faithfulness de 0,76 de la Tabla 31 corresponde al generador local, el escenario más exigente, y que con el generador de Groq el mismo sistema es sensiblemente más fiel al contexto que recupera. Es, además, una conclusión útil para decidir cómo desplegar el sistema: el generador es el componente que más condiciona la fidelidad de las respuestas, por encima de la recuperación.

La generación local mostró durante la evaluación un segundo fallo, más grave. En la pregunta sobre el pleno del 26-01-2023, con un contexto de varias proposiciones parecidas del mismo día, Qwen2.5-7B entró en un bucle de repetición –un fallo conocido de los modelos de lenguaje con decodificación determinista, como la que se usa aquí con temperatura 0 [22]–: empezó a enumerar los puntos del orden del día y, a partir del quinto, repitió el mismo bloque una y otra vez. Como el generador local no tenía un límite de longitud de salida, siguió generando durante 50 minutos (más de 52 000 *tokens*, frente a los ~1 700 de la respuesta legítima más larga de la batería), y el proceso parecía colgado. El registro del servidor de Ollama permitió ver que no lo estaba. Se corrigió fijando un límite de 4 096 *tokens* de salida, como ya tenían el generador de Groq y el del GraphRAG. Con el límite, la respuesta termina en unos 4 minutos y el post-procesado elimina los bloques repetidos, pero sigue siendo una respuesta deficiente (Faithfulness 0,50): el límite impide que el fallo bloquee la aplicación, pero no evita que el modelo pequeño entre en bucle. Se ha mantenido así en la evaluación, porque es exactamente lo que obtendría un usuario con el generador local.

8.6. Comparativa GraphRAG frente a RAG vectorial

Los dos sistemas se evaluaron con metodologías distintas a propósito, no por omisión. Se valoró aplicar RAGAS también a la narración final del GraphRAG –tratando las filas

devueltas por el SPARQL como contexto-, pero se descartó: el cuello de botella real del GraphRAG es la generación de la consulta SPARQL, ya cubierta con precisión de ejecución frente a una verdad conocida (Sección 3.2.2), no la fidelidad de una narración que parte de datos ya correctos. Medir la narración con un juez LLM no habría apuntado al factor que de verdad limita a ese sistema.

En la práctica, cada sistema resuelve mejor un tipo de pregunta distinto. El GraphRAG es la opción adecuada para preguntas analíticas sobre el corpus completo –conteos, rankings, comparaciones entre grupos o a lo largo de los años– porque puede agregar sobre miles de proposiciones con una única consulta y devolver un número exacto y verificable. Es la misma limitación de la recuperación vectorial que motiva el trabajo de Edge et al.: recuperar fragmentos sueltos no permite responder preguntas globales dirigidas al corpus completo [3]. Su propuesta la resuelve con resúmenes de comunidades del grafo generados por un LLM, mientras que este trabajo lo hace con consultas SPARQL sobre datos estructurados, pero el diagnóstico de partida es el mismo. El RAG vectorial es la opción adecuada para preguntas abiertas o narrativas sobre un tema o una sesión concreta –“qué se dijo sobre...”– porque puede citar y resumir el contenido real de las actas, algo que el GraphRAG, al operar solo sobre los datos estructurados del grafo, no puede hacer.

8.7. Configuración final y valoración global

La configuración final del RAG vectorial usa Groq `gpt-oss-120b` como generador principal, con caída automática a Ollama Qwen2.5-7B-Instruct local si la cuota de Groq se agota o el servicio no responde, y con un límite de 4096 *tokens* de salida para el generador local; `nomic-embed-text` como modelo de embeddings; recuperación híbrida de tres canales, con selección de palabras clave por rareza en el canal literal, y reordenamiento Cohere `rerank-multilingual-v3.0`. La del GraphRAG usa Qwen3:8B local para generar el SPARQL y Groq (con el mismo respaldo local) para narrar el resultado.

En conjunto, el RAG vectorial muestra una recuperación de contexto prácticamente completa (Context Relevance 0,99 sobre 50 preguntas) y una fidelidad de generación que depende sobre todo del generador: 0,76 con el modelo local de 7B y en torno a 0,84 con `gpt-oss-120b` en el experimento controlado, con patrones de fallo identificados (lectura errónea de “decae”, mezcla entre bloques). El GraphRAG, por su parte, muestra una precisión de ejecución verificada frente a datos reales en las preguntas analíticas que el vectorial no puede resolver bien. Ambos resultados son consistentes con el diseño arquitectónico del Capítulo 6: la recuperación híbrida y el reordenamiento existen específicamente para sostener la fidelidad de la generación, y la separación en dos sistemas existe específicamente para que cada tipo de pregunta se responda con la herramienta adecuada.

8.8. Limitaciones

- **Variabilidad del juez.** Un juez basado en un LLM no es determinista, una fuente de ruido conocida de esta metodología [21]. Se midió al volver a puntuar las mismas 22 respuestas del generador local en el experimento controlado (la única diferencia con la primera puntuación era la eliminación del listado de fuentes que se añade al final de la respuesta): la Faithfulness de una misma pregunta cambió de media 0,16 entre las dos pasadas, y hasta 1,00 en un caso extremo. El cambio medio, en cambio,

fue prácticamente nulo (+0,003): el ruido se compensa al promediar. Por eso los resultados se dan siempre como media y desviación típica sobre toda la batería, y la puntuación de una pregunta aislada no debe interpretarse por sí sola. Por el mismo motivo, en el experimento controlado las dos condiciones se puntuaron en la misma pasada.

- **Evaluación principal con el generador local.** Las métricas de la Tabla 31 corresponden a Qwen2.5-7B. Con `gpt-oss-120b` solo se midió el subconjunto de 22 preguntas del experimento controlado, y sobre los contextos anteriores a la corrección de la recuperación: la cuota gratuita diaria de Groq no permite redactar las 50 preguntas en una sola jornada.
- **Fallos del generador local no eliminables.** Los patrones de la Sección 8.5 y el bucle de repetición son límites de un modelo de 7B. El límite de salida impide que bloquee la aplicación, y el cambio de generador reduce mucho los patrones de error, pero no se ha resuelto ninguno de los dos de forma garantizada con el modelo local.
- **Batería de referencia.** Las 50 preguntas son suficientes para que las medias sean estables frente al ruido del juez, pero no cubren todos los tipos de consulta posibles, y la elección de temas con pocas proposiciones –necesaria para verificar las respuestas de referencia completas– deja fuera los temas más amplios del corpus, en los que la respuesta de referencia no puede ser exhaustiva.

9. Conclusiones y trabajo futuro

9.1. Conclusiones del trabajo

9.2. Cumplimiento de objetivos

9.3. Conclusiones sobre las pruebas realizadas

9.4. Contribuciones científico-técnicas

9.5. Trabajo futuro

Bibliografía

- [1] *Ley 19/2013, de 9 de diciembre, de transparencia, acceso a la información pública y buen gobierno*. Boletín Oficial del Estado, núm. 295, de 10 de diciembre de 2013 (BOE-A-2013-12887). <https://www.boe.es/buscar/act.php?id=BOE-A-2013-12887>
- [2] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., y Kiela, D. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020. https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf
- [3] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., y Larson, J. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. arXiv:2404.16130, 2024. <https://arxiv.org/abs/2404.16130>
- [4] OpenAI. *Introducing ChatGPT*. 30 de noviembre de 2022. <https://openai.com/index/chatgpt/>
- [5] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., y Fung, P. *Survey of Hallucination in Natural Language Generation*. ACM Computing Surveys, vol. 55, núm. 12, art. 248, 2023. <https://doi.org/10.1145/3571730>
- [6] Hogan, A., Blomqvist, E., Cochez, M., D'Amato, C., De Melo, G., Gutierrez, C., Kirrane, S., Labra Gayo, J. E., Navigli, R., Neumaier, S., Ngonga Ngomo, A. C., Polleres, A., Rashid, S. M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., y Zimmermann, A. *Knowledge Graphs*. ACM Computing Surveys, vol. 54, núm. 4, art. 71, pp. 1–37, 2021. <https://doi.org/10.1145/3447772>
- [7] W3C. *RDF 1.1 Concepts and Abstract Syntax*. W3C Recommendation, 25 de febrero de 2014. <https://www.w3.org/TR/rdf11-concepts/>
- [8] Hitzler, P., Krötzsch, M., Parsia, B., Patel-Schneider, P. F., y Rudolph, S. (eds.) *OWL 2 Web Ontology Language Primer (Second Edition)*. W3C Recommendation, 11 de diciembre de 2012. <https://www.w3.org/TR/owl2-primer/>
- [9] W3C. *SPARQL 1.1 Query Language*. W3C Recommendation, 21 de marzo de 2013. <https://www.w3.org/TR/sparql11-query/>
- [10] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., y Wang, H. *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv:2312.10997, 2023. <https://arxiv.org/abs/2312.10997>
- [11] Nogueira, R., y Cho, K. *Passage Re-ranking with BERT*. arXiv:1901.04085, 2019. <https://arxiv.org/abs/1901.04085>

- [12] Giuliani, A., Manca, M. M., Piano, L., et al. *Are LLMs Adequate SPARQL Query Generators? Investigating Zero-Shot NL-to-SPARQL Translation*. Neural Computing and Applications, vol. 38, art. 33, 2026. <https://link.springer.com/article/10.1007/s00521-025-11799-x>
- [13] Hernandez-Camero, I. V., Garcia-Lopez, E., Garcia-Cabot, A., y Caro-Alvaro, S. *Context-Aware Few-Shot Learning SPARQL Query Generation from Natural Language on an Aviation Knowledge Graph*. Machine Learning and Knowledge Extraction, vol. 7, núm. 2, art. 52, 2025. <https://doi.org/10.3390/make7020052>
- [14] Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., y Zhou, J. *Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation*. Proceedings of the VLDB Endowment (PVLDB), vol. 17, núm. 5, pp. 1132–1145, 2024. <https://doi.org/10.14778/3641204.3641221>
- [15] Li, H., Zhang, J., Liu, H., Fan, J., Zhang, X., Zhu, J., Wei, R., Pan, H., Li, C., y Chen, H. *CodeS: Towards Building Open-source Language Models for Text-to-SQL*. Proceedings of the ACM on Management of Data (SIGMOD 2024), vol. 2, núm. 3, pp. 1–28, 2024. <https://doi.org/10.1145/3654930>
- [16] Unger, C., Böhmann, L., Lehmann, J., Ngonga Ngomo, A.-C., Gerber, D., y Cimiano, P. *Template-based Question Answering over RDF Data*. En *Proceedings of the 21st International Conference on World Wide Web (WWW 2012)*, pp. 639–648, 2012. <https://doi.org/10.1145/2187836.2187923>
- [17] Dirección General de Trabajo. *Resolución de 4 de abril de 2025 por la que se registra y publica el XIX Convenio colectivo estatal de empresas de consultoría, tecnologías de la información y estudios de mercado y de la opinión pública*. Boletín Oficial del Estado (BOE-A-2025-7766), 2025. https://www.boe.es/diario_boe/txt.php?id=BOE-A-2025-7766
- [18] Dirección General de Trabajo. *Resolución de 13 de abril de 2026 por la que se registra y publica el Acta de la Comisión paritaria por la que se aprueba la actualización de las tablas salariales del XIX Convenio colectivo estatal de empresas de consultoría, tecnologías de la información y estudios de mercado y de la opinión pública*. Boletín Oficial del Estado (BOE-A-2026-9024), 2026. https://www.boe.es/diario_boe/txt.php?id=BOE-A-2026-9024
- [19] Es, S., James, J., Espinosa-Anke, L., y Schockaert, S. *RAGAs: Automated Evaluation of Retrieval Augmented Generation*. En *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL 2024)*, pp. 150–158, 2024. <https://aclanthology.org/2024.eacl-demo.16/>
- [20] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., y Stoica, I. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. En *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, Datasets and Benchmarks Track, 2023. <https://arxiv.org/abs/2306.05685>
- [21] Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., et al. *A Survey on LLM-as-a-Judge*. arXiv:2411.15594, 2024. <https://arxiv.org/abs/2411.15594>

- [22] Holtzman, A., Buys, J., Du, L., Forbes, M., y Choi, Y. *The Curious Case of Neural Text Degeneration*. En *International Conference on Learning Representations (ICLR 2020)*, 2020. <https://arxiv.org/abs/1904.09751>